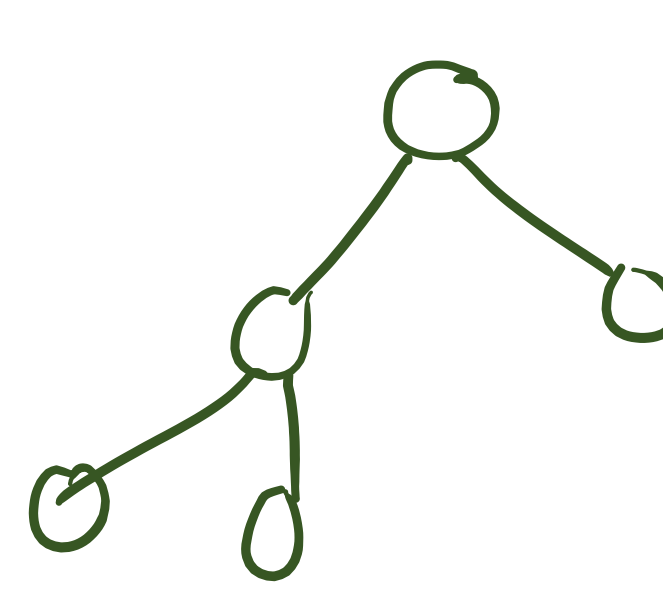
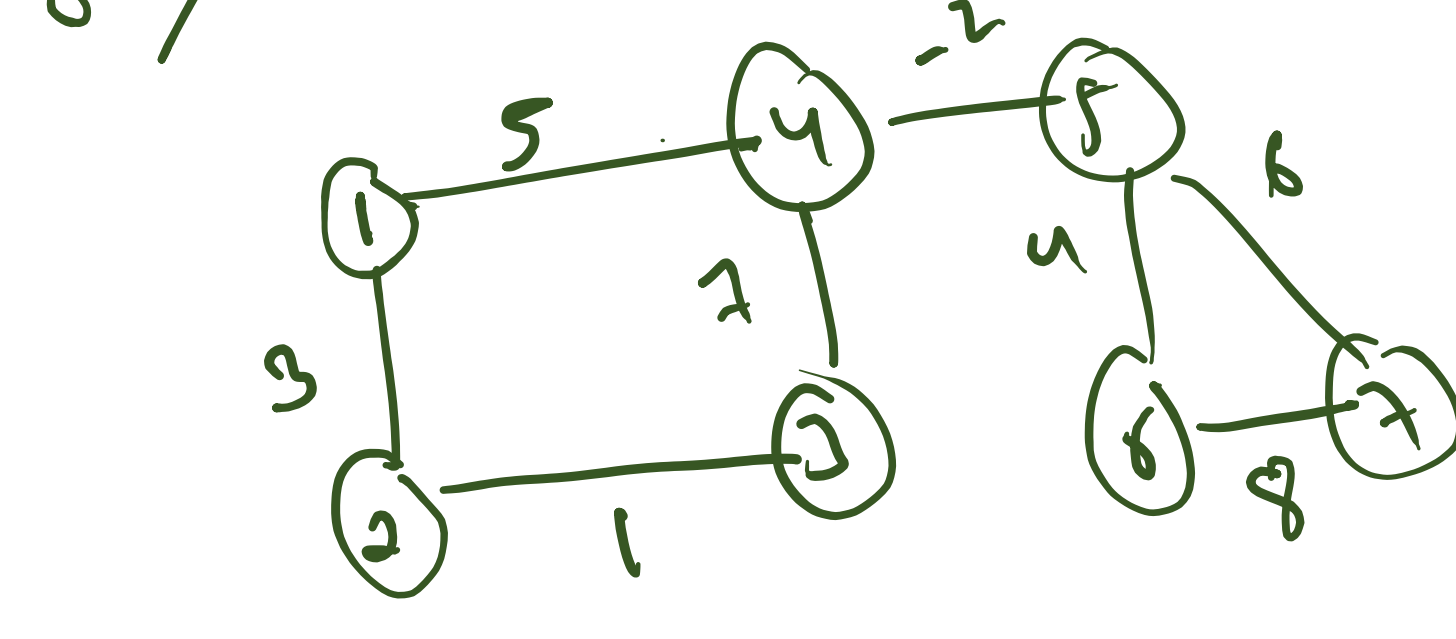
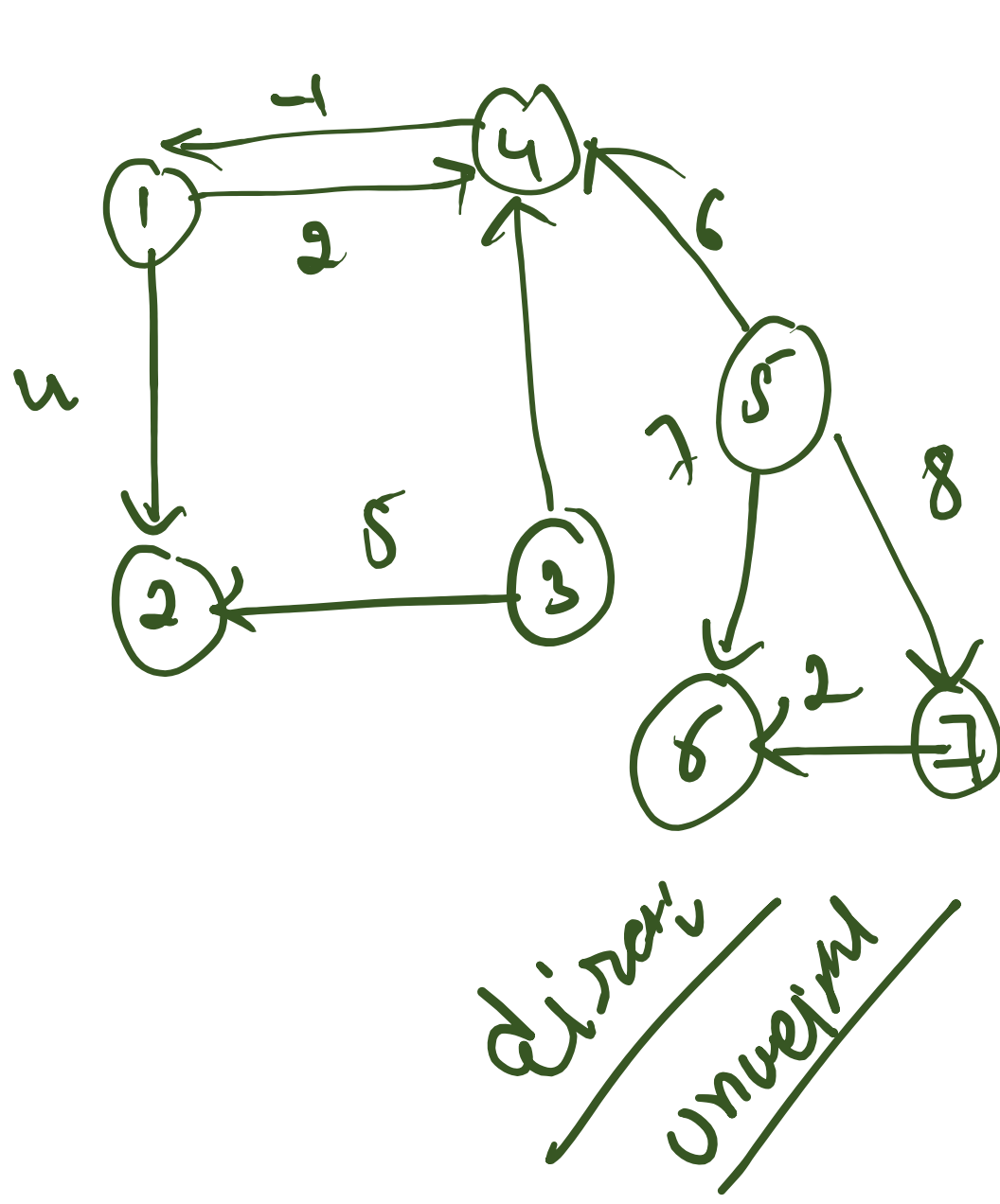
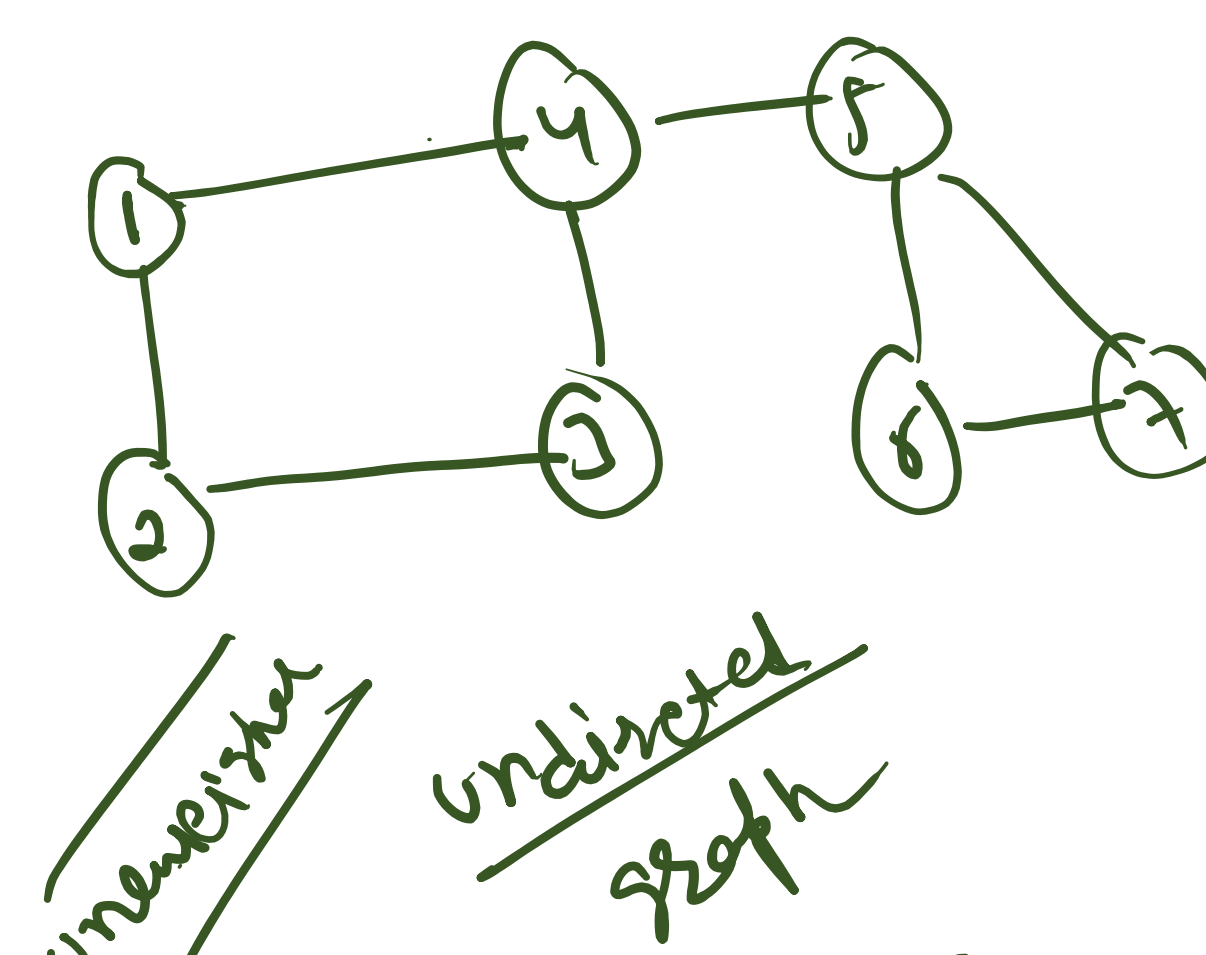
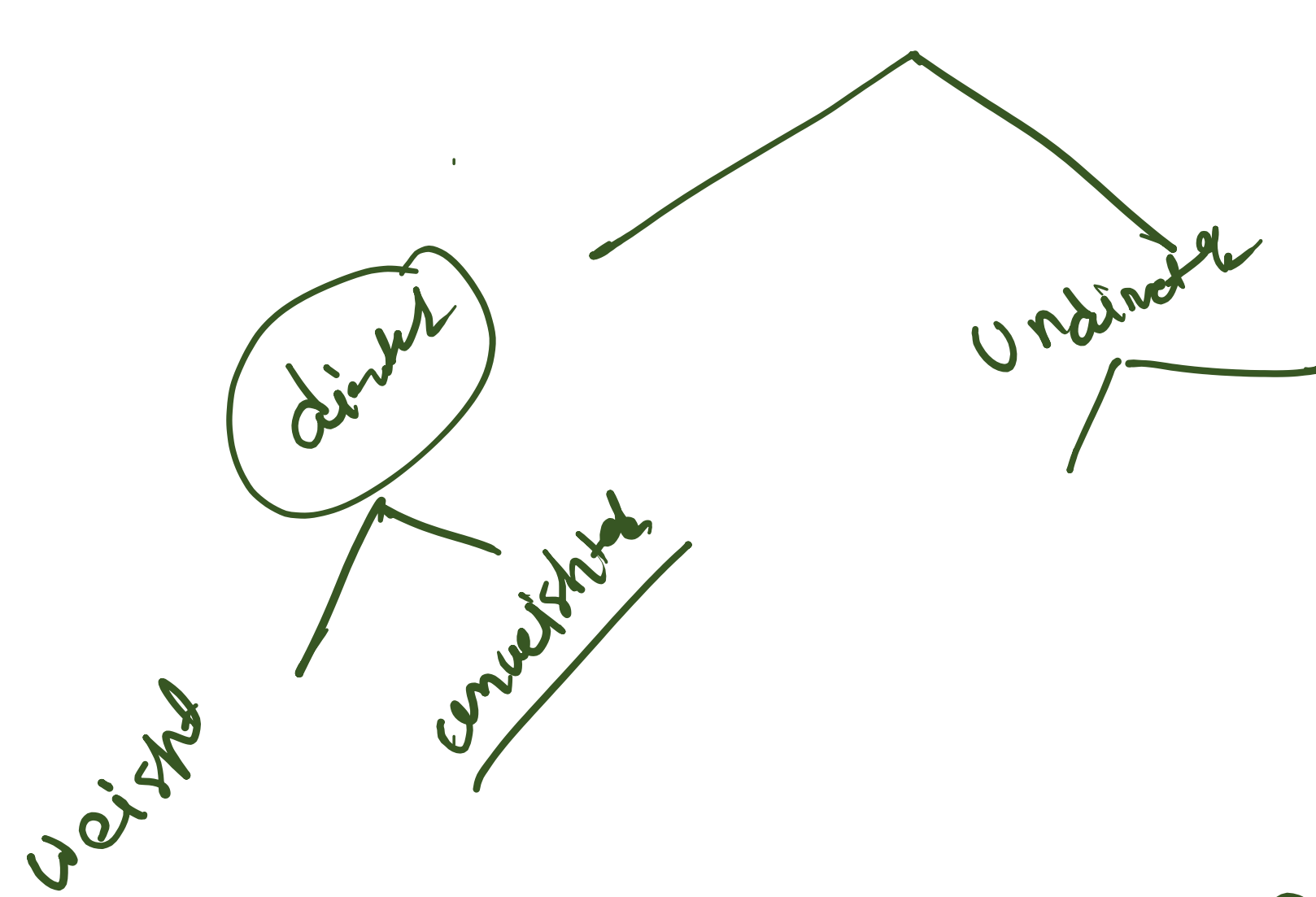
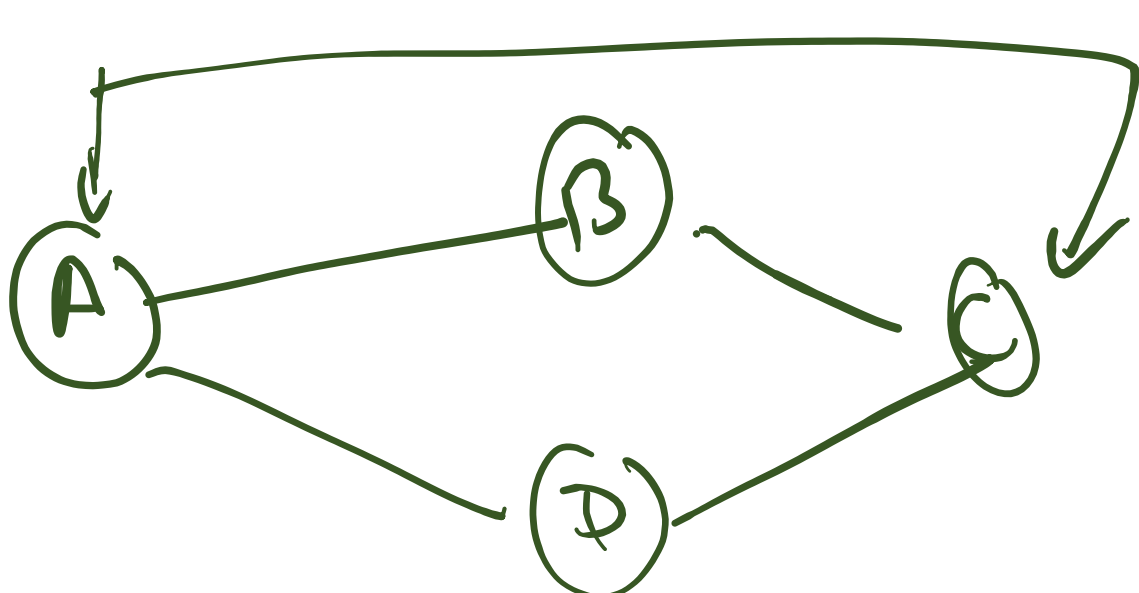


Tree $\rightarrow$ graph
graph $\rightarrow$ Tree X

Graph
 $\rightarrow$ It is finite set of edges and $V+E$

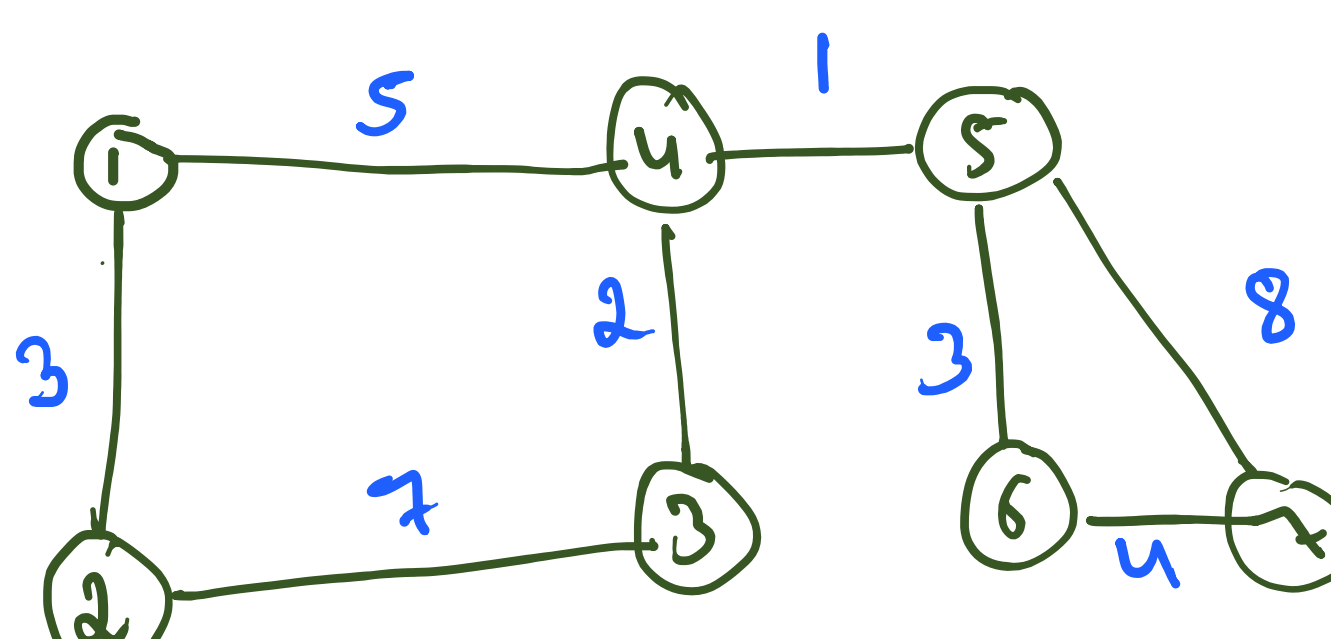


Uset



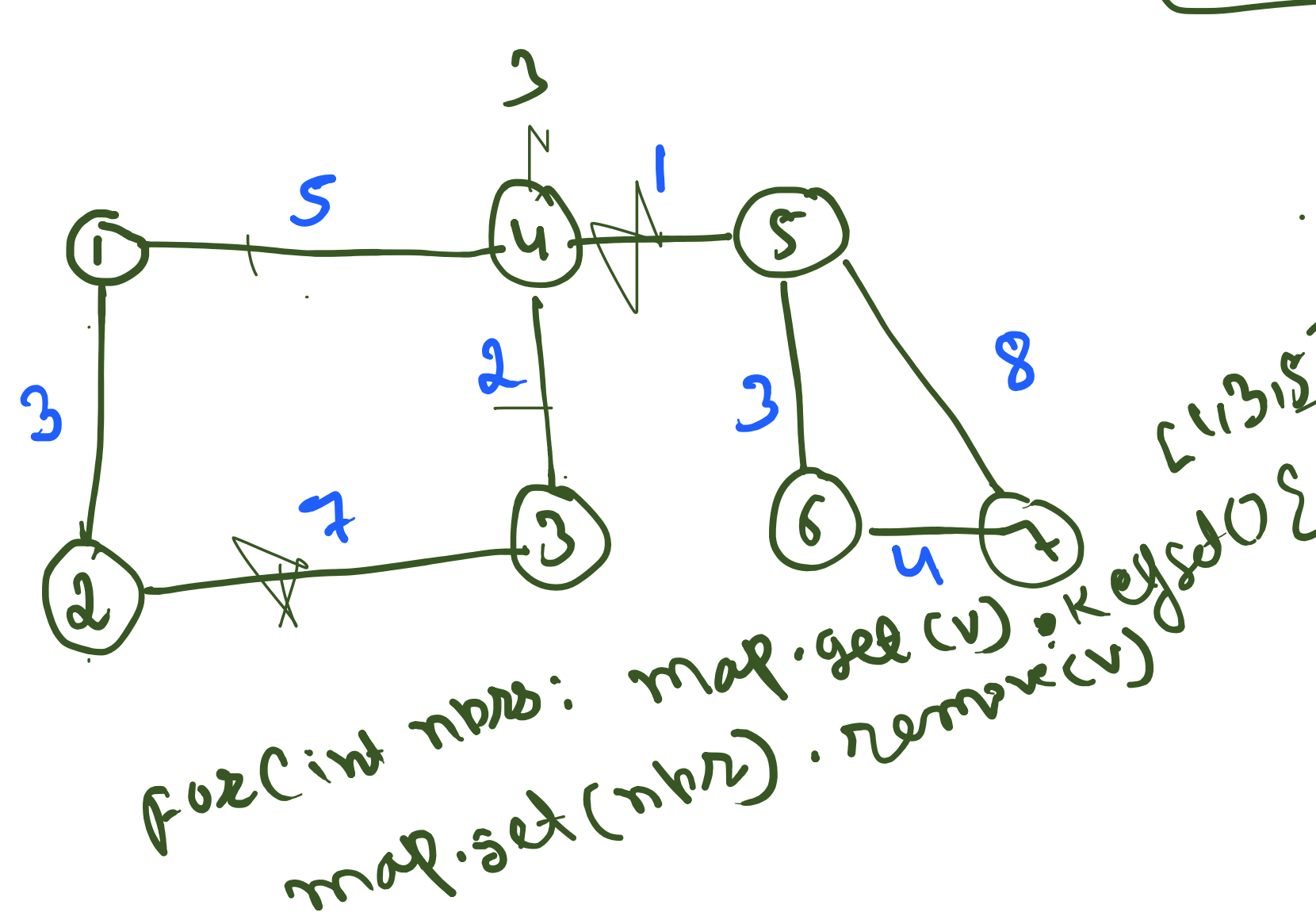
Adj List

1	2, 3, 5
2	1, 3, 7
3	2, 7, 4, 5
4	3, 5, 7
5	1, 3, 4
6	
7	



Adj matrix

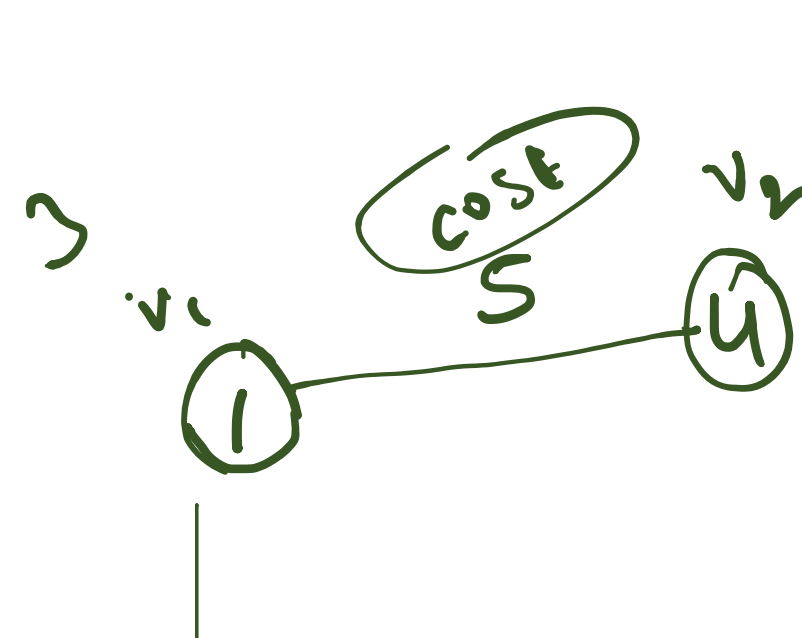
	1	2	3	4	5	6	7
1	0	3	0	5	0	0	0
2	3	0	7	0	0	0	0
3	0	7	0	2	0	0	0
4	5	0	2	0	1	0	0
5	0	2	2	1	0	3	8
6	0	0	0	0	3	0	4
7	0	0	0	0	8	4	0



for (int nbrs : map.get(v).keySet())
map.set(nbrs, map.get(nbrs).remove(v))

1	2K
2	3K
3	4K
4	5K
5	6K
6	7K
7	8K

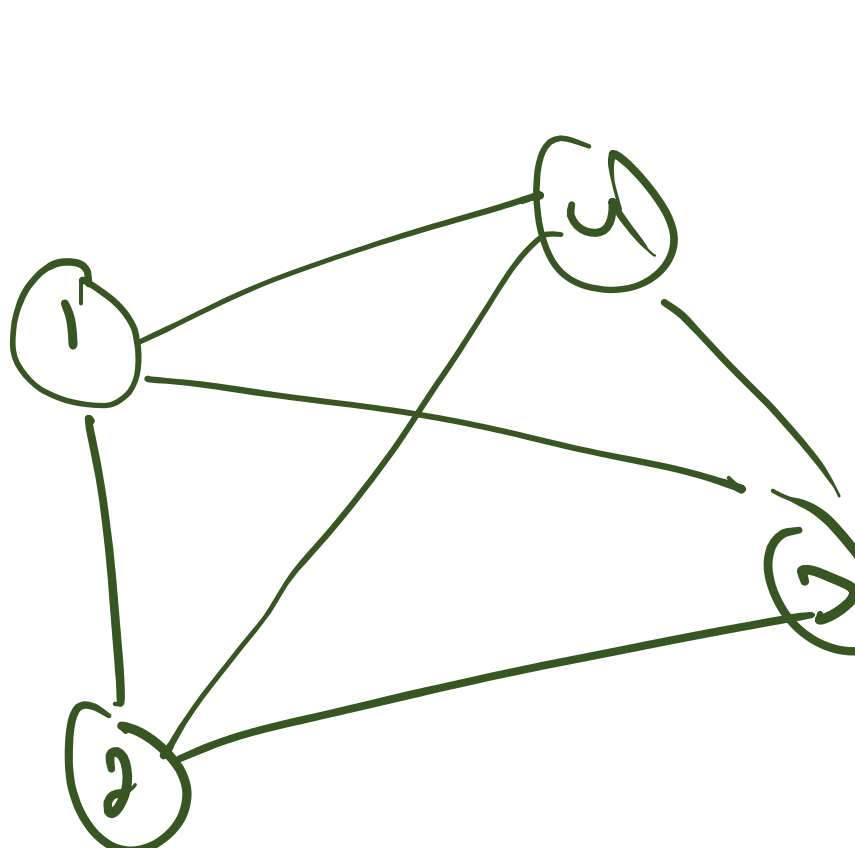
2, 3	1, 3
4, 5	3, 7
2, 7	1, 5
4, 2	3, 2
4, 1	5, 1
6, 3	
7, 8	7, 4
8, 8	3, 3
8, 4	7, 4



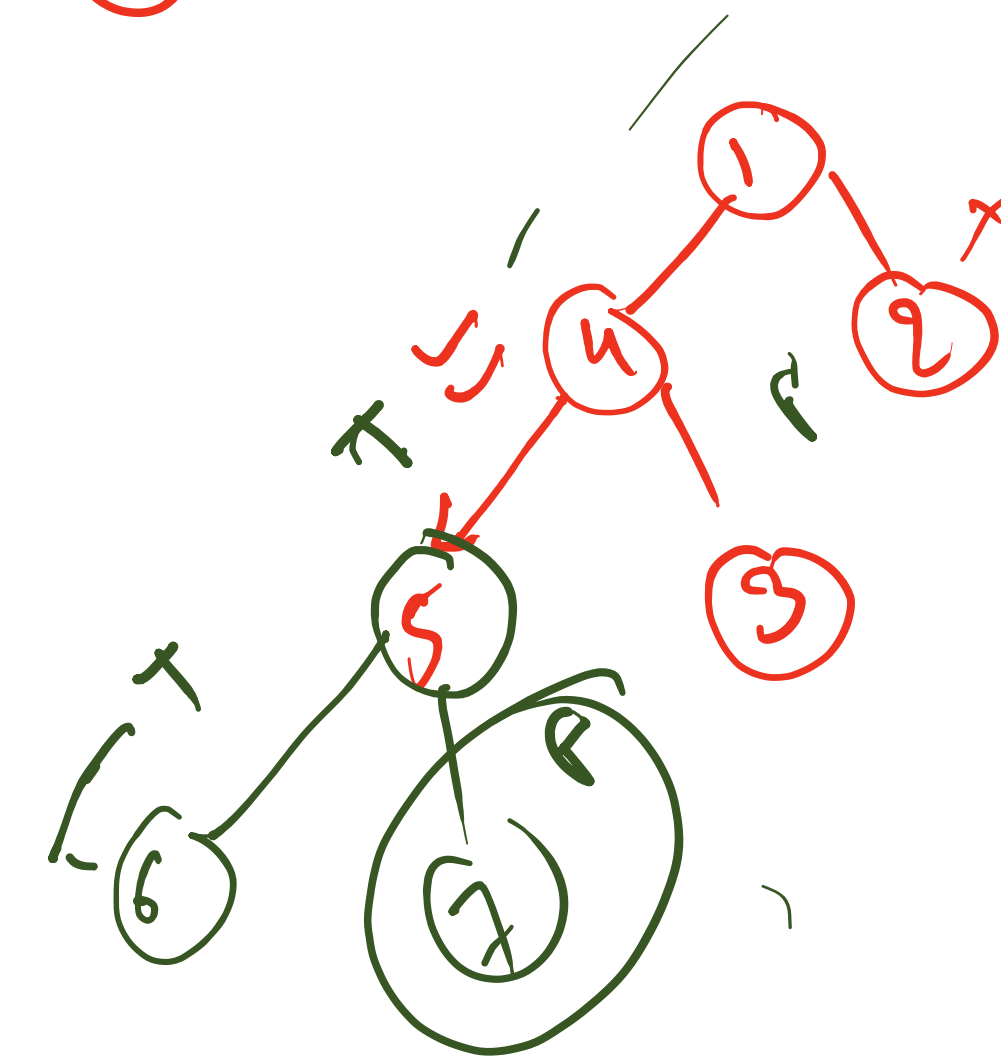
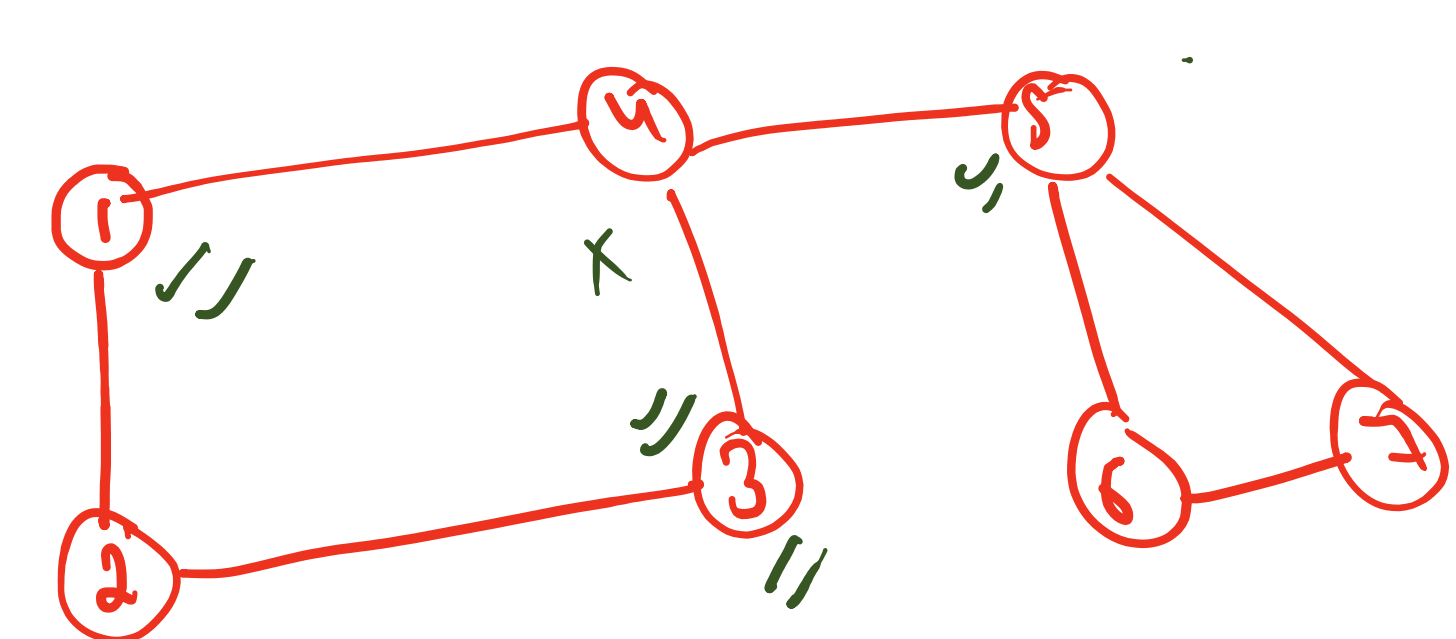
map.get(v).put(v, cost)
map.set(v2, put(v1, cost))

1	1K
2	2K
3	3K
4	4K

n(v)



$$\frac{n \times (n-1)}{2}$$



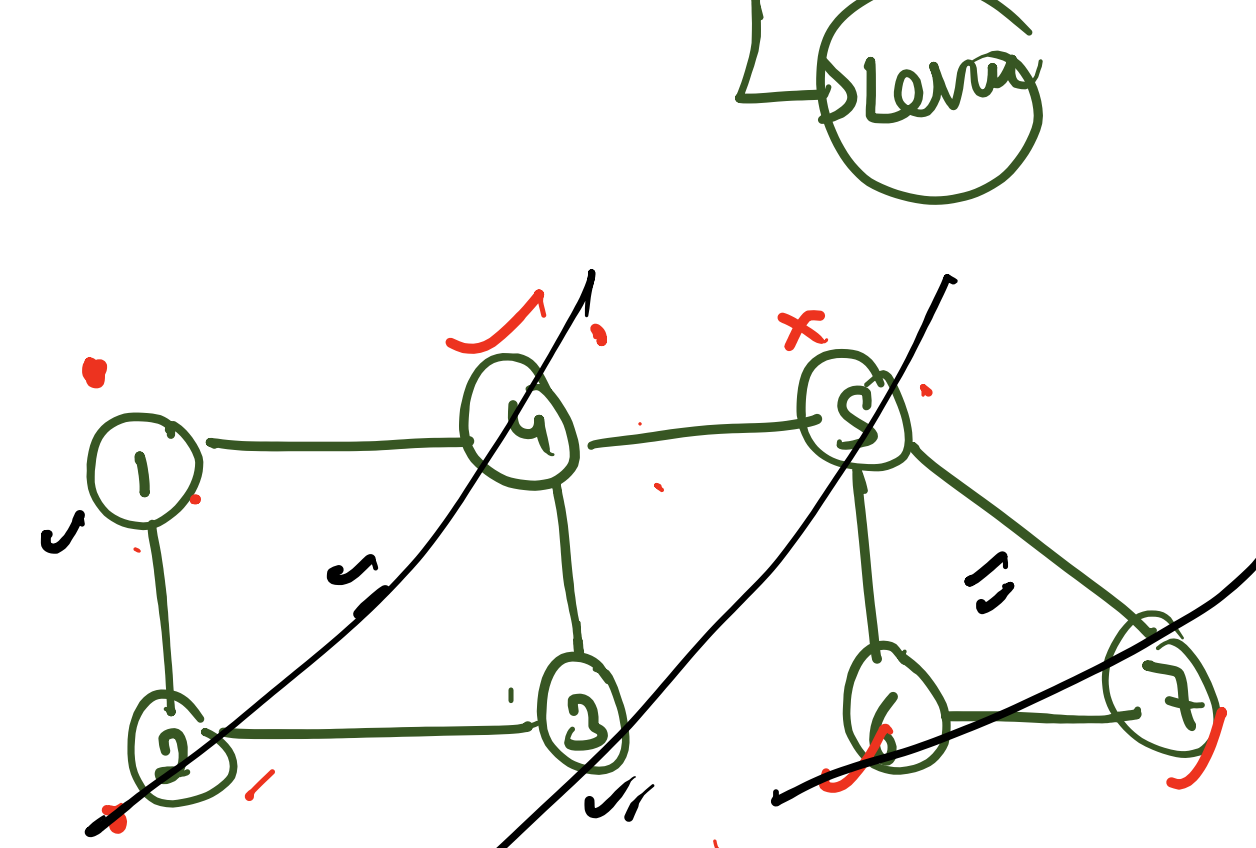
```
public boolean hasPath(int src, int des, HashSet<Integer> visited) {
    if (src == des) return true;
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            boolean ans = hasPath(nbrs, des, visited);
            if (ans) return ans;
        }
    }
    return false;
}
```



4, 6, 7
(6, 6)
[1, 5, 3]
(5, 6)
[4, 2]
(4, 6)
(1, 6)

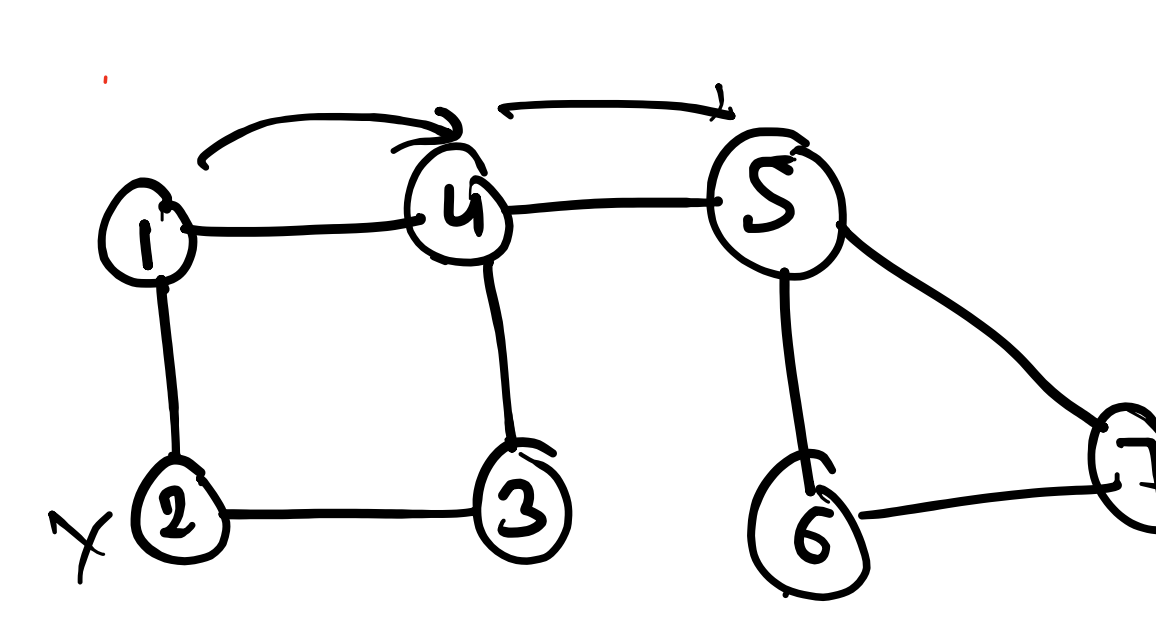
BFS

DFS

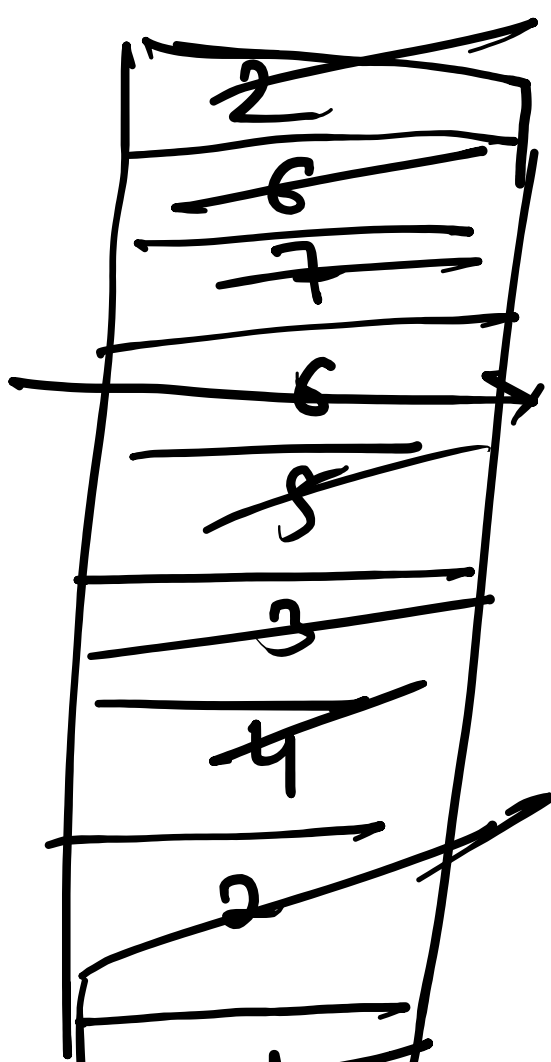


1, 2, 4
3, 5, 6
7

1. remove
2. ignore
3. visited mark
4. self work
5. Add unvisited nbrs



1, 4, 5
7, 6, 3
2



1. remove
2. ignore
3. visited
4. self work
5. Add nbrs